AeroqAI

AI Application Evaluation, RAG Optimization and Quality Assurance Platform

Synopsis submitted to

Shri Ramdeobaba College of Engineering & Management, Nagpur in partial fulfillment of requirement for the award of the degree of

Bachelor of Technology (B.Tech)

In

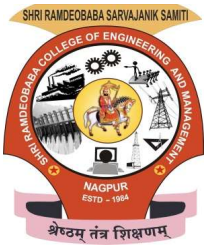
COMPUTER SCIENCE AND ENGINEERING
(Cyber Security)

By

Aditi Bhattad(Roll no.03)
Rudra Baiswal(Roll no.48)
Sachin Yadav (Roll no.49)
Shreyash karemore(Roll no.55)

Guide

Prof. Monika Bagade



Department of Computer Science and Engineering — Cyber Security
Shri Ramdeobaba College of Engineering & Management, Nagpur 440 013
(An Autonomous Institute affiliated to Rashtrasant Tukdoji Maharaj Nagpur University Nagpur)

August 2026

Abstract

The rapid adoption of Large Language Models (LLMs) and Retrieval-Augmented Generation (RAG) has enabled intelligent applications such as question-answering systems, enterprise chatbots, document assistants, and AI-powered information retrieval platforms. However, ensuring the **accuracy, reliability, consistency, and efficiency** of these applications remains challenging because their performance depends on multiple interconnected components, including the language model, prompt design, document chunking, embedding model, retrieval parameters, reranking, and generation settings.

AeroqAI is a proposed intelligent AI quality-assurance platform designed to automate the **evaluation, optimization, experimentation, and regression testing** of LLM/RAG applications. The platform will evaluate AI-generated responses using multiple quality dimensions such as correctness, relevance, faithfulness, safety, latency, and cost. It will allow developers to systematically compare different models and RAG configurations and identify configurations that provide improved overall performance.

The proposed system will integrate an **AI Evaluation Engine, RAG Configuration Optimizer, Experiment Tracking Module, Regression Detection Engine, and RAG Debugger**. The Experiment Tracking module will maintain configurations, datasets, metrics, and execution results to support reproducibility and meaningful comparison of experiments. The Regression Detection module will compare a new version against a validated baseline and identify quality degradation before deployment. The RAG Debugger will provide visibility into retrieved context, retrieval results, prompts, and generated responses, helping developers investigate potential causes of incorrect or unsupported answers.

By integrating **evaluation, optimization, experiment tracking, debugging, and quality-regression detection** into a unified platform, AeroqAI aims to transform AI application testing from a predominantly manual process into a **systematic, measurable, reproducible, and engineering-driven quality-assurance workflow** for modern LLM/RAG applications.

Introduction

The rapid advancement of Large Language Models (LLMs) and Retrieval-Augmented Generation (RAG) has enabled intelligent applications such as enterprise chatbots, document assistants, question-answering systems, and AI-powered information retrieval platforms. However, ensuring the accuracy, reliability, and consistency of these applications remains challenging because their performance depends on several interconnected factors, including the language model, prompt design, document chunking, embeddings, retrieval parameters, reranking, and generation settings.

Evaluating LLM/RAG applications is also difficult because AI-generated responses are often non-deterministic and cannot always be evaluated using traditional exact-match testing. A response may be semantically correct despite different wording, while a fluent response may contain incorrect or unsupported information. Furthermore, manually testing different models and RAG configurations makes it difficult to identify the best configuration, reproduce experiments, and detect quality degradation.

Another important challenge is quality regression. A change in a model, prompt, or retrieval configuration may cause an AI application to perform worse even though the application continues to function normally. Traditional software-testing and CI/CD processes do not adequately address this AI-specific quality problem.

AeroqAI is proposed to address these challenges through an integrated platform for LLM/RAG evaluation, optimization, experiment tracking, regression detection, and debugging. The platform aims to provide a systematic and measurable approach for evaluating AI applications, identifying better-performing configurations, investigating incorrect responses, and detecting quality degradation before deployment.

Problem Definition

The rapid adoption of Large Language Models (LLMs) and Retrieval-Augmented Generation (RAG) has enabled the development of intelligent applications such as question-answering systems, enterprise chatbots, document assistants, and AI-powered information retrieval platforms. However, ensuring that these applications consistently produce accurate, relevant, and reliable responses remains a major challenge. The quality of an LLM/RAG application depends on several interconnected components, including the language model, prompt design, document chunking strategy, embedding model, retrieval parameters, reranking, and generation settings. Changes to any of these components may improve one aspect of the system while negatively affecting another, making systematic evaluation essential.

Evaluating LLM/RAG applications is fundamentally different from testing conventional software because AI-generated responses can vary in wording while still being correct. Conversely, a response may appear natural and convincing while containing incorrect or unsupported information. In RAG applications, it is therefore necessary to evaluate not only the final generated answer but also the relevance, quality, and usefulness of the retrieved information. This creates a need for evaluation methods that can assess multiple dimensions of AI response quality rather than relying only on fixed inputs and expected outputs.

Another major challenge is the lack of systematic experimentation, reproducibility, and regression detection. Developers often manually test different models, prompts, and RAG configurations, which becomes difficult as the number of combinations increases. An AI application may still function normally while its response quality decreases after changes to the model, prompt, retrieval strategy, or knowledge base. Traditional software testing and CI/CD processes may not detect these AI-specific quality regressions, increasing the risk of deploying a version that performs worse than the previously validated baseline.

Therefore, there is a need for an integrated platform that can automatically evaluate AI applications, optimize RAG configurations, track experiments, investigate response quality, and detect regressions before deployment. **AeroqAI** addresses this need by providing a unified AI quality-assurance platform for evaluating, experimenting, optimizing, debugging, and regression testing LLM/RAG applications. It helps developers compare configurations, identify effective RAG settings, understand response-quality issues, and maintain consistent AI application performance throughout development and deployment.

Problem Objectives

The primary objective of AeroqAI is to design and develop an intelligent platform for the systematic evaluation, experimentation, optimization, and quality assurance of LLM and RAG-based applications. The proposed system aims to reduce the dependency on manual testing and provide developers with a structured approach to measuring, improving, and maintaining AI application quality.

The major objectives of the proposed project are:

1.**To develop a basic Retrieval-Augmented Generation (RAG) pipeline** that processes documents, divides them into meaningful chunks, generates embeddings, retrieves relevant information, and produces context-aware responses using an LLM.

2.**To develop an automated evaluation engine** for measuring the quality of generated responses using metrics such as correctness, faithfulness, relevance, and other configurable quality measures.

3.**To implement an automated RAG configuration optimization mechanism** that evaluates different combinations of parameters, including chunk size, Top-K retrieval, embedding models, reranking strategies, prompts, and generation settings, and identifies better-performing configurations based on predefined quality and performance criteria.

4.**To develop an experiment-tracking mechanism** that records configurations, datasets, evaluation metrics, latency, token usage, estimated cost, timestamps, and other relevant execution details to support reproducibility and comparison.

5.**To implement AI quality-regression detection** by comparing a newly evaluated application version with a previously validated baseline and identifying significant degradation in defined quality metrics.

6.**To provide configurable AI quality gates** that generate clear PASS or FAIL decisions based on predefined evaluation and regression thresholds before a new version proceeds toward deployment.

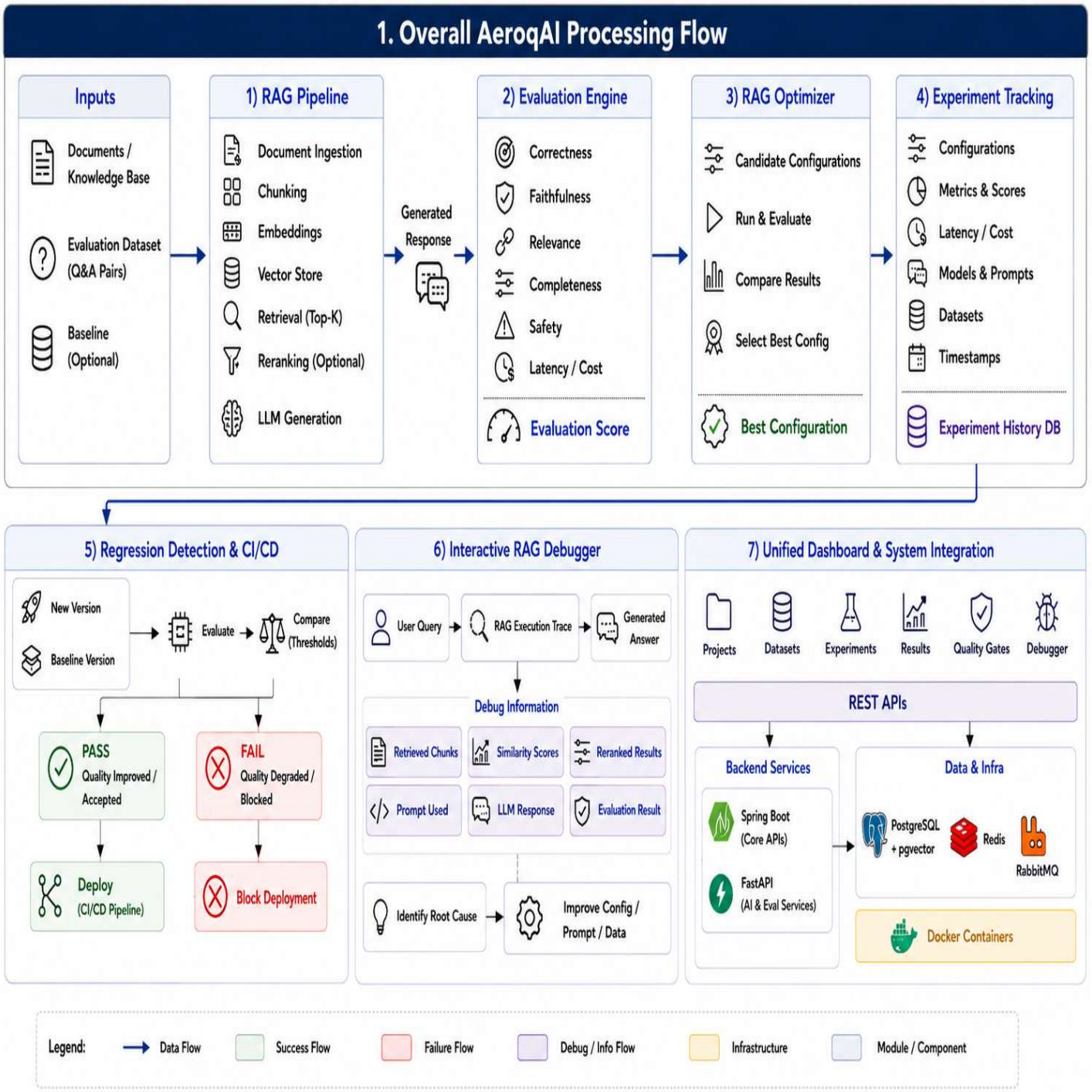
7.**To develop an interactive RAG-debugging mechanism** that captures important stages of the retrieval and generation pipeline, providing visibility into retrieved context, retrieval results, prompts, and generated responses to help identify possible causes of incorrect or unsupported answers.

8.**To integrate the developed components into a unified web-based platform** with a centralized dashboard for managing projects, datasets, configurations, experiments, evaluation results, and regression reports.

9.**To support CI/CD integration by enabling automated AI evaluation** and quality checks throughout the development and deployment lifecycle.

Proposed Plan of Work

The proposed project will be developed using a **modular and incremental approach**, where each major component of AeroqAI will be designed, implemented, and tested independently before being integrate into the complete platform. This approach will make the system easier to develop, test, maintain, and extend. The development will begin with the core RAG pipeline and gradually incorporate evaluation, optimization, experiment tracking, regression detection, debugging, and dashboard capabilities.



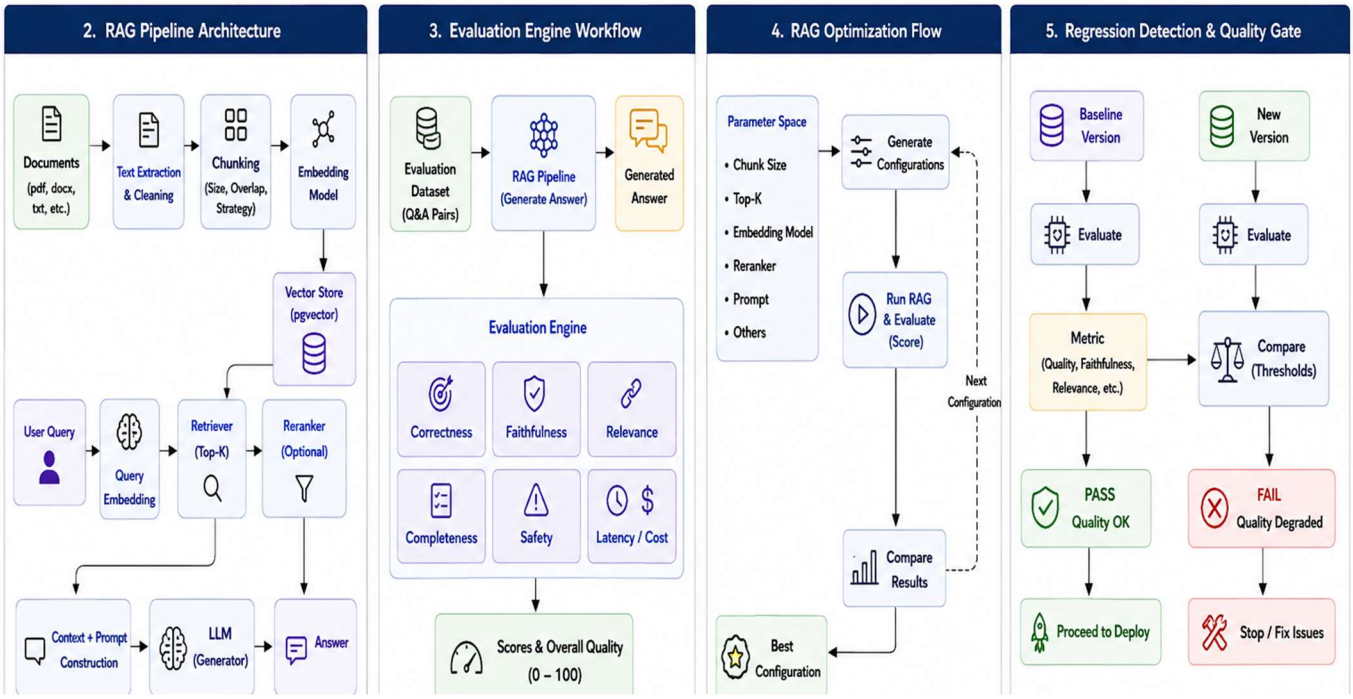


Fig1. Overall AeroqAI Processing Flow

Module 1 — RAG Pipeline

This module builds the basic RAG pipeline for AeroqAI. A sample knowledge base is divided into smaller text chunks and stored as vector representations in a vector database. When a user asks a question, the system finds the most relevant information and provides it to the LLM to generate a context-based answer. The details of the query, retrieved information, and generated response are recorded for further evaluation. response are recorded for further evaluation.

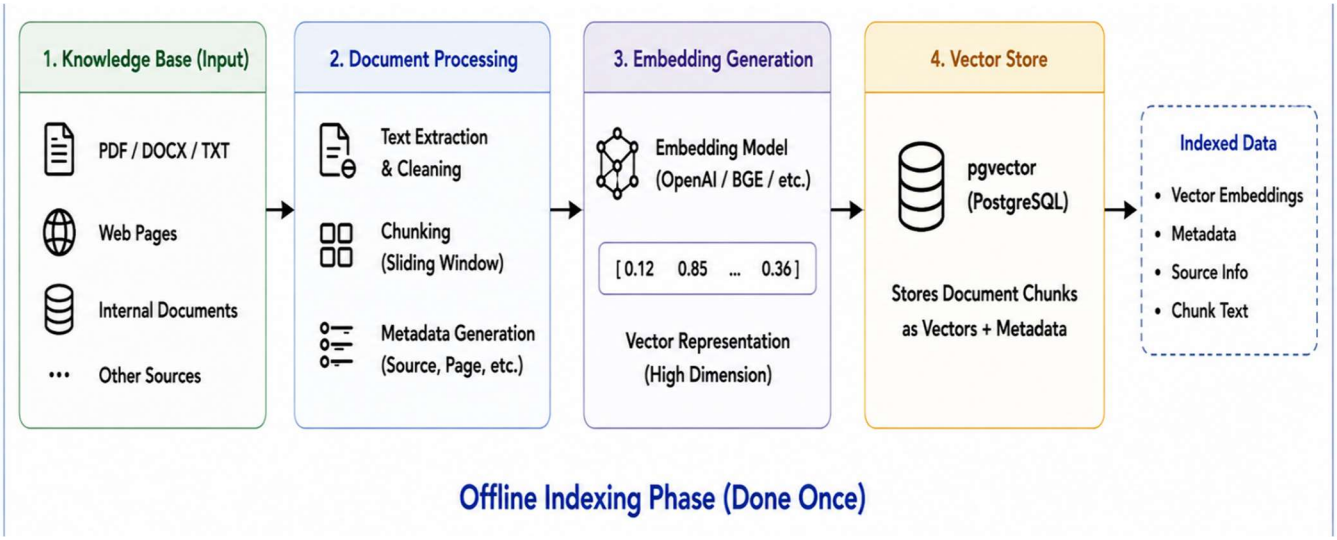


Fig 2.Offline Indexing Phase

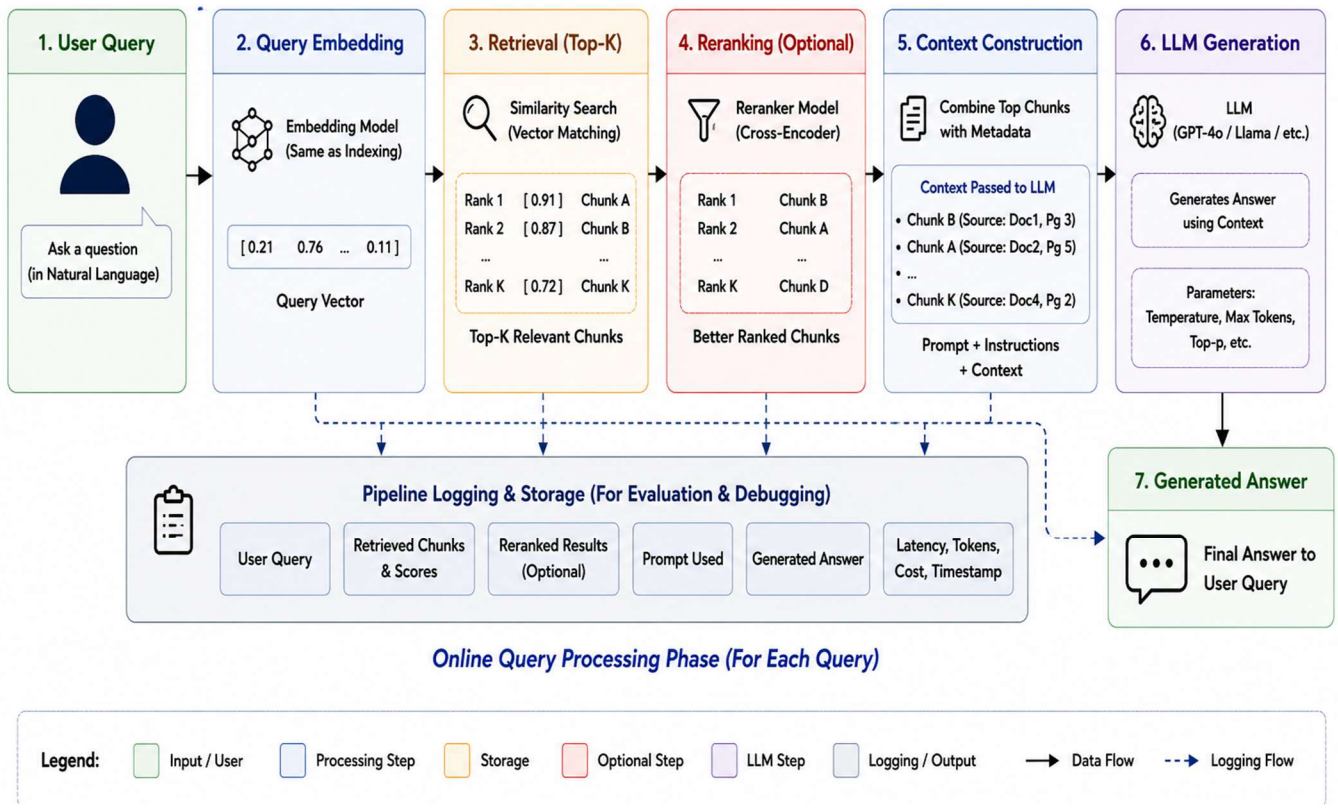


Fig3. RAG Pipeline Architecture Daigram

Module 2 — Evaluation Engine

This module checks how well the AI system is performing. A set of questions and expected answers is used to evaluate the generated responses based on factors such as correctness, faith fulness, relevance, completeness, and safety. This helps measure response quality without depending only on exact word-to-word matching

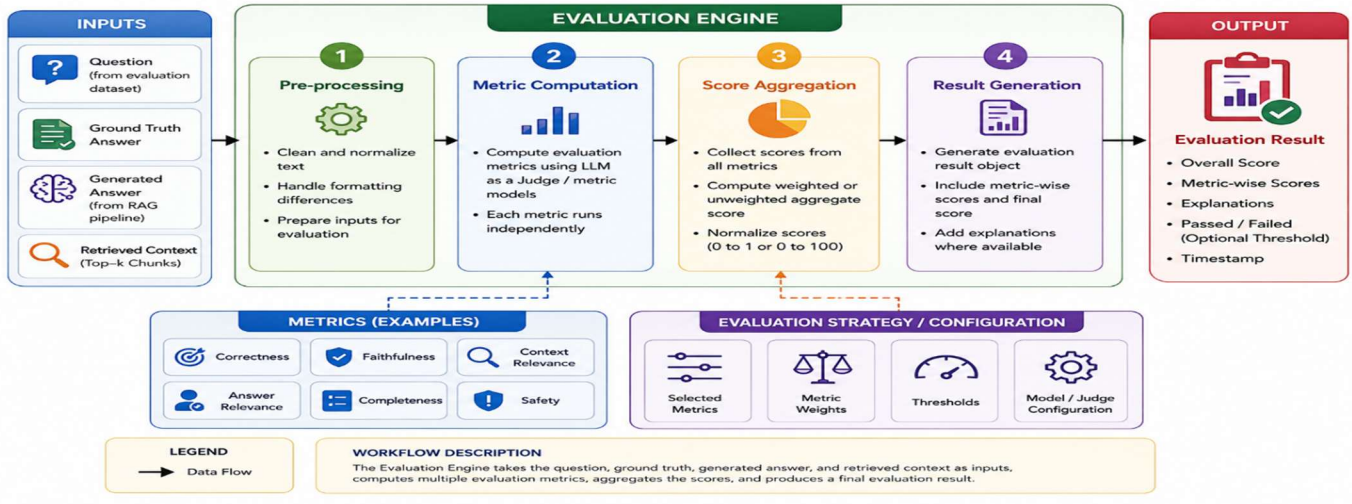


Fig 4.Evaluation Engine Working

Module 3 — RAG Optimizer

This module helps find better RAG configurations by trying different settings such as chunk size, Top-K, embedding models, reranking, and prompts. Each configuration is tested using the same evaluation dataset, and the results are compared to identify the configuration that performs best.

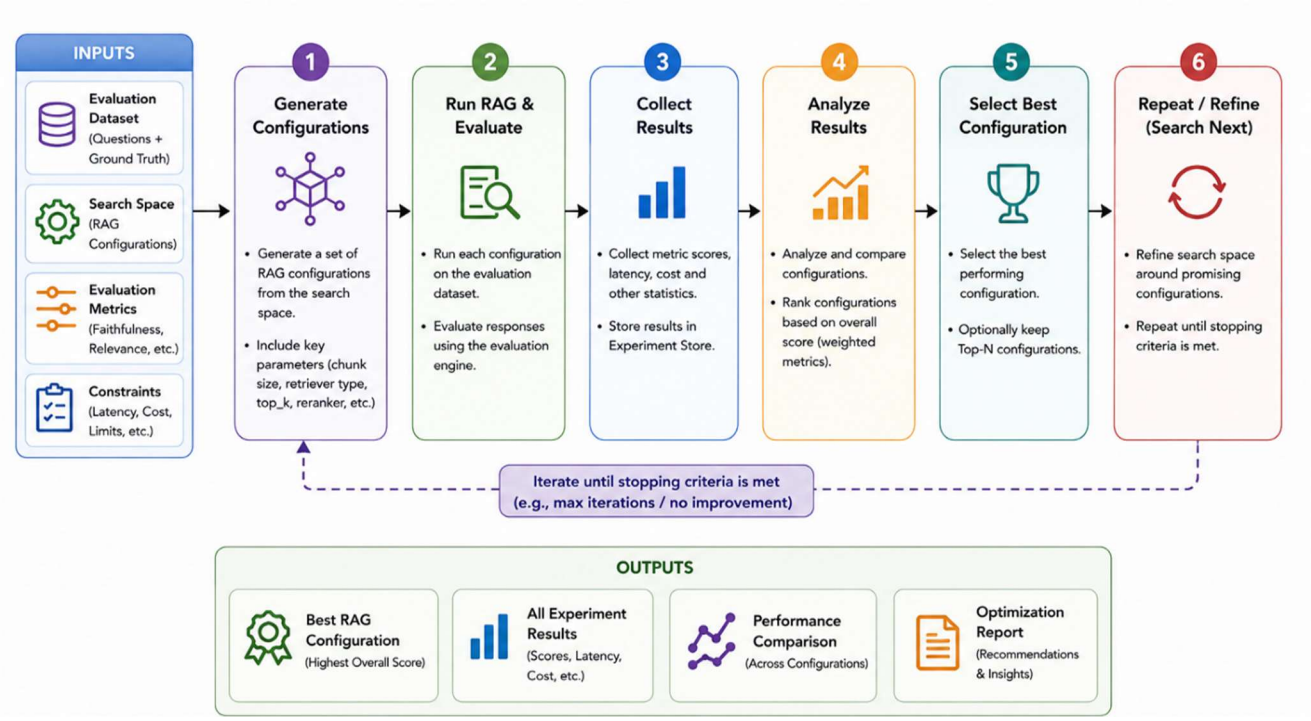


Fig 5.RAG Optimizer Flow

Module 4 — Experiment Tracking

This module keeps a clear record of the experiments performed during development. It stores important details such as the configuration used, evaluation scores, response time, token usage, cost, dataset, and timestamp. This makes it easier to compare experiments, understand the effect of changes, and reproduce earlier results.

Module 5 — Quality Regression Detection and CI/CD

This module checks whether a new version of the AI application performs better or worse than a previously validated version. The system compares the new evaluation results with a defined baseline and measures changes in important performance metrics such as quality, relevance, faithfulness, safety, latency, and cost.

Based on predefined thresholds and conditions, the module identifies any significant degradation in performance and generates a clear **PASS or FAIL** result. This helps developers detect AI quality issues early and provides an automated quality check that can be integrated into the CI/CD workflow before deployment.

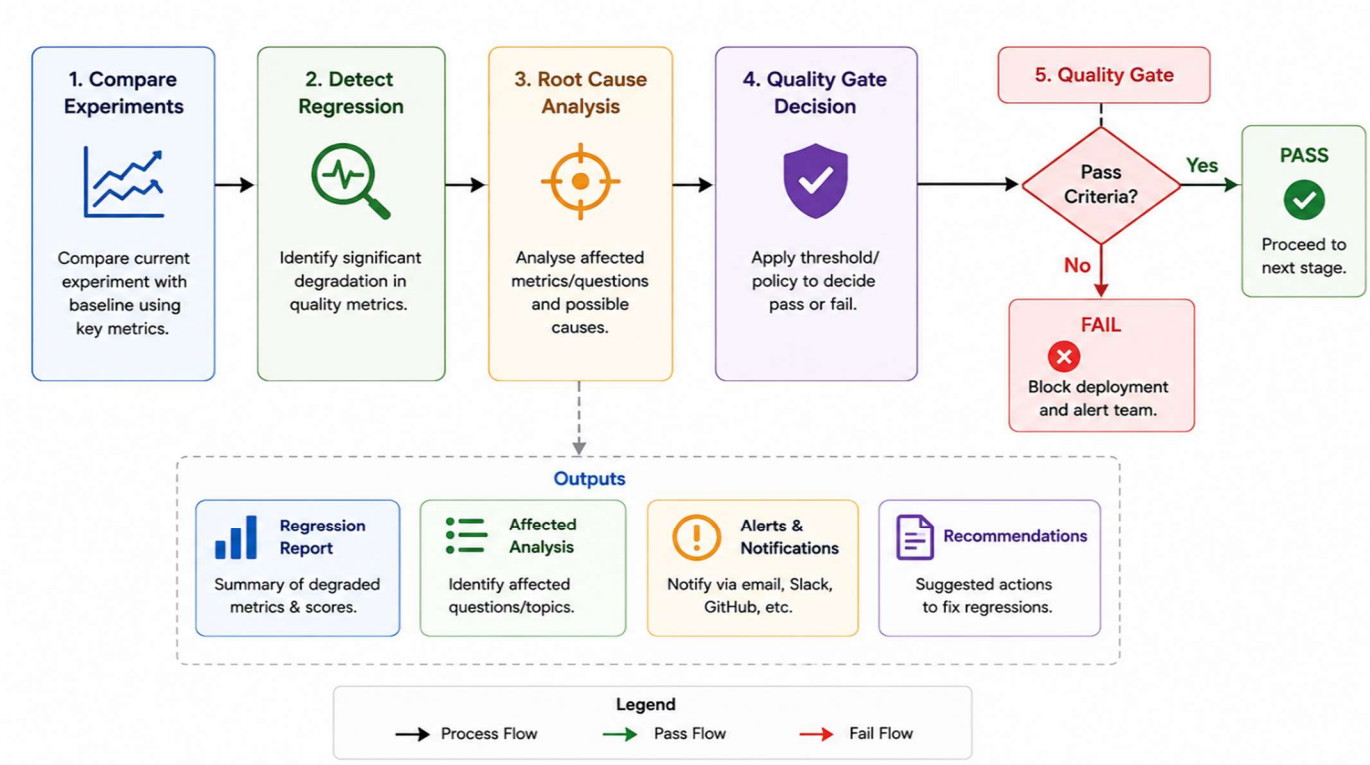


Fig 6.Regression Detection and Quality Gate

Module 6 — Interactive RAG Debugger

This module helps developers understand why a RAG system gives an incorrect answer. It records the complete journey of a query, including the retrieved documents, similarity scores, reranking, prompt, generated answer, and evaluation results. By examining this information, developers can identify whether the problem occurred during retrieval, reranking, prompting, or answer generation.

Module 7 — System Integration

This module brings all the individual components together into one web-based platform. It provides a common interface for managing projects and datasets, running evaluations and experiments, comparing results, checking regressions, and investigating RAG responses. The final dashboard will present these results in a simple and understandable way for developers.

Methodology

The proposed AeroqAI platform will be developed using a **modular and iterative methodology**. The system will be built step by step, starting with the preparation of the knowledge base and evaluation dataset, followed by RAG pipeline development, automated evaluation, configuration optimization, experiment tracking, regression detection, and debugging. Each stage will be tested individually before being integrated into the complete platform.

Step 1 — Dataset and Knowledge Base Preparation

The first step involves preparing the knowledge base and evaluation dataset that will be used throughout the project. The knowledge base may contain documents such as HR policies or technical documentation, while the evaluation dataset will contain questions, expected answers, and optional reference contexts.

For example:

Question: How many annual leave days do standard employees receive?
Expected Answer: Standard full-time employees receive 20 days of paid annual leave per year.

The same evaluation dataset will be used across different experiments so that the performance of various RAG configurations can be compared fairly.

Step 2 — Document Processing

The selected documents will be converted into machine-readable text and cleaned to remove unnecessary content. The processed text will then be divided into smaller chunks using configurable chunk sizes and overlap values. This prepares the documents for efficient storage and retrieval**.**

Document → Text Extraction → Cleaning → Chunking → Text Chunks

Step 3 — Embedding Generation

Each text chunk will be converted into a numerical vector using an embedding model. These vectors represent the semantic meaning of the text and allow the system to find information that is relevant to a query based on meaning rather than only matching exact words.

Step 4 — Vector Storage and Retrieval

The generated embeddings will be stored in **pgvector**, which will be used for similarity-based retrieval. When a user submits a question, the query will also be converted into a vector. The system will compare the query with the stored vectors using similarity search and select the **Top-K most relevant chunks** to provide context for answer generation.

Step 5 — Answer Generation

The retrieved information will be combined with the user's question and passed to the selected LLM. The model will use this context to generate an answer. During this process, AeroqAI will record relevant execution details such as the query, retrieved chunks, configuration, generated response, and response latency.

Step 6 — Automated Evaluation

The generated responses will then be evaluated using the prepared evaluation dataset. The evaluation engine will measure important quality aspects such as **correctness, faithfulness,**

relevance. These scores will provide a consistent basis for comparing different experiments and configurations.

Step 7 — Configuration Optimization

Multiple RAG configurations are generated from selected hyperparameters, for example:

```
Chunk Size = [250, 500, 1000]
Top-K      = [3, 5, 10]
Reranker   = [Enabled, Disabled]
```

Each configuration is evaluated on the same test dataset, and results are compared to identify the configuration with the best overall performance.

Once the evaluation process is established, AeroqAI will test different RAG configurations to determine which settings provide better results. Parameters such as **chunk size, Top-K retrieval, and reranking** can be varied systematically.

Step 8 — Experiment Tracking

Every evaluation and optimization run will be recorded as a separate experiment. The system will store the configuration used along with its evaluation results and other relevant information. This will allow developers to compare experiments, understand the effect of configuration changes, and reproduce earlier results when required.

Step 9 — Regression Detection

A selected experiment acts as the baseline. A new version is evaluated on the same dataset, and a configurable threshold determines whether it passes or fails, for example:

```
Baseline Quality = 91%
New Quality      = 85%
Allowed Drop     = 5%
Quality Drop     = 6%   =>  Result = FAIL
```

Step 10 — Debugging and Integration

The final stage focuses on understanding and integrating the complete system. AeroqAI will maintain the execution details of RAG queries, including retrieved information, configuration, generated responses, and evaluation results, so that developers can investigate possible causes of poor responses and identify areas for improvement.

Once the individual modules have been developed and tested, they will be connected through APIs and integrated into a **unified web-based dashboard**. The final platform will bring together RAG execution, evaluation, optimization, experiment tracking, regression detection, and debugging within a single workspace. This integration will provide a centralized environment for managing and monitoring the overall AI application quality.

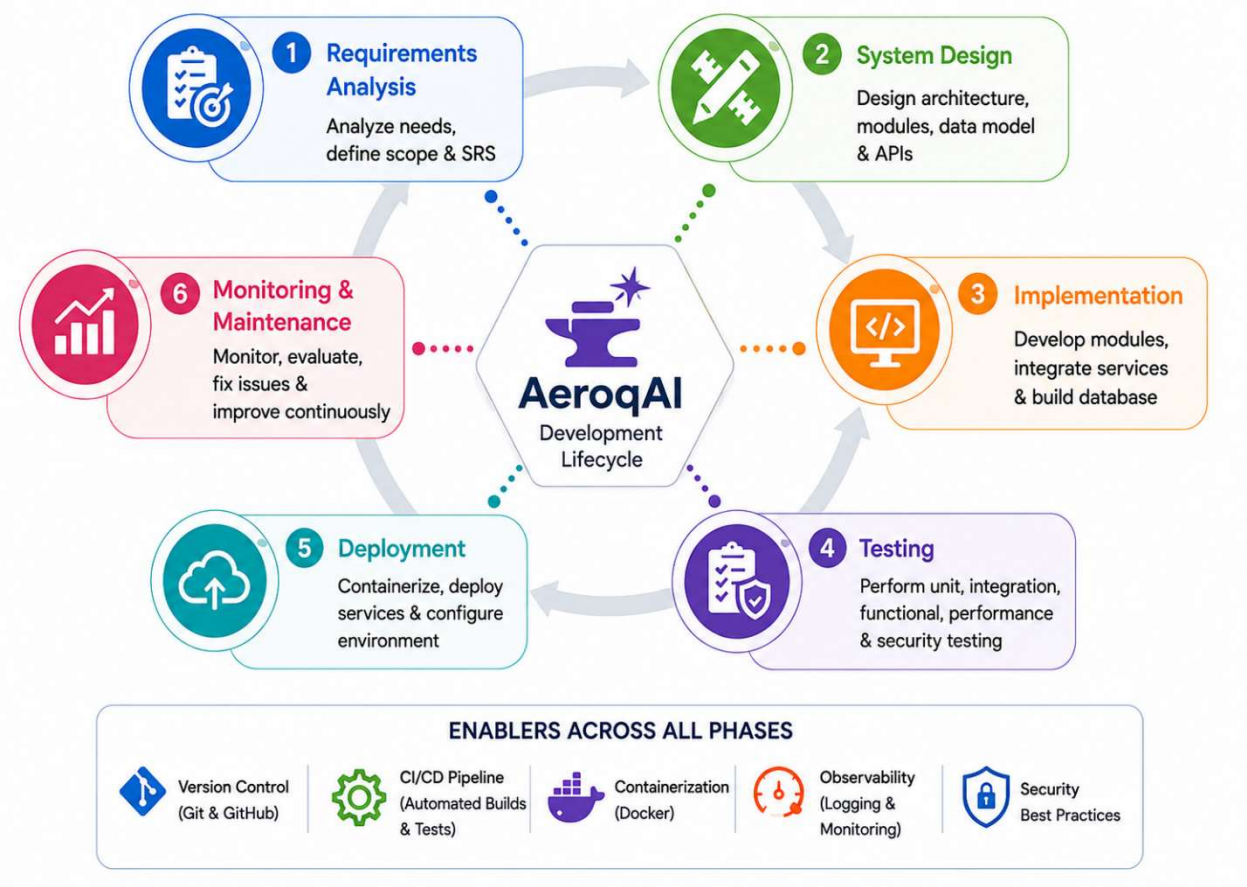


Fig 7.AeroqAI Methodology

Technology Stack

AeroqAI follows a **two-service backend architecture**, where each service is responsible for the type of workload it is best suited to handle. **Java with Spring Boot** manages the core application and production-level responsibilities, including user management, authentication, projects, experiments, REST APIs, policies, queues, and CI/CD operations. **Python with FastAPI** handles the AI-focused components, such as RAG processing, embedding generation, LLM evaluation, and LLM-as-a-Judge functionality.

This separation keeps the system organized and makes the architecture easier to maintain and scale. It also allows each technology to be used where it provides the most value. The two services communicate through well-defined APIs, enabling smooth interaction between the application and AI components. Together, they work as a single unified platform for AI application evaluation and quality assurance. The overall tech stack is designed to support **reliability, scalability, flexibility, and efficient AI processing**. It also provides a strong foundation for integrating additional AI models, evaluation methods, and deployment tools in the future.

Layer	Technology	Purpose
Frontend	Next.js, TypeScript, Tailwind CSS	Dashboards for projects, experiments, evaluation results, configuration comparison, and debugging traces.
Backend	Java 21, Spring Boot, REST APIs	Manages projects, datasets, experiments, users, authentication, and communication with other services.
AI / Evaluation Layer	Python, FastAPI, LLM APIs, Embedding models	Handles RAG processing, embeddings, LLM-as-a-Judge evaluation, and optimization.
Database	PostgreSQL	Stores application data, projects, datasets, experiments, configurations, and evaluation results.
Vector Database	pgvector	Stores embeddings and performs vector similarity search within PostgreSQL.
Caching / Async Processing	Redis, RabbitMQ	Redis caches frequently accessed data; RabbitMQ supports asynchronous experiment and evaluation workloads.
Containerization	Docker	Provides a consistent development and deployment environment for all services.
CI/CD	GitHub Actions	Executes automated evaluation workflows and quality gates on application changes.
Monitoring	OpenTelemetry	Traces application and evaluation workflows across the integrated system.

Fig 8. Tech stack

AeroqAI uses a modern technology stack for AI evaluation, RAG processing, experimentation, and deployment. The **frontend** uses **Next.js, TypeScript, and Tailwind CSS** for dashboards, while the **backend** uses **Java 21, Spring Boot, and REST APIs** for users, projects, datasets, experiments, authentication, and service communication. The **AI and Evaluation Layer** uses **Python, FastAPI, LLM APIs, and embedding models** for RAG processing, embeddings, evaluation, and optimization.

PostgreSQL and **pgvector** manage application data and vector search, while **Redis and RabbitMQ** support caching and asynchronous workloads. **Docker**, **GitHub Actions**, and **OpenTelemetry** support deployment, CI/CD automation, quality gates, monitoring, and tracing. Together, these technologies provide a **scalable, maintainable, and reliable platform for AI application evaluation and quality assurance**.

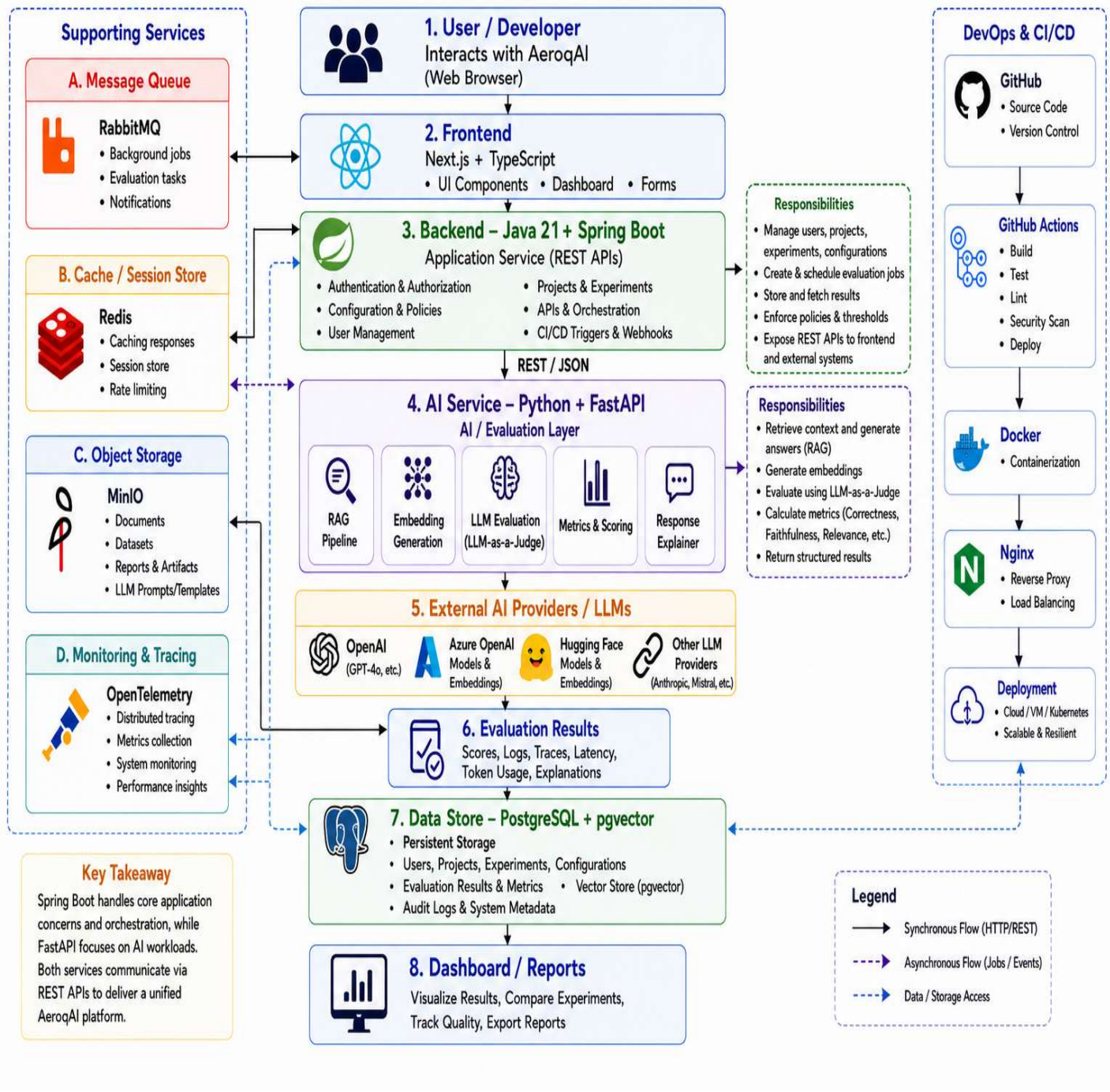


Fig. 8. Technology and interactive diagram

1.Frontend Technologies

Next.js, TypeScript, and Tailwind CSS

The frontend of AeroqAI will provide a web-based interface through which users can manage projects, datasets, experiments, evaluations, and debugging results. Next.js will be used to build the application structure, while TypeScript will provide type safety and improve maintainability. Tailwind CSS will be used to create a responsive and consistent user interface.

2. Application Backend

Java 21 and Spring Boot

The main application backend will be developed using Java 21 and Spring Boot. It will handle application-level responsibilities such as user management, authentication, project and experiment management, REST APIs, configuration management, policies, and communication with supporting services.

Spring Boot is suitable for this layer because AeroqAI requires a structured backend capable of handling multiple modules, database operations, API communication, background tasks, and integration with external services.

3.AI and Evaluation Service

Python and FastAPI

The AI-specific functionality will be implemented as a separate Python service using FastAPI. This service will handle RAG processing, embedding generation, LLM interaction, automated evaluation, LLM-as-a-Judge, and metric calculation.

Separating AI workloads from the main application backend allows AeroqAI to use Python's AI and machine-learning ecosystem while keeping the core application services independently manageable.

4.RAG and Vector Processing

Embeddings and pgvector

AeroqAI uses embeddings to represent documents and queries as numerical vectors. These representations allow the system to perform semantic similarity search and retrieve relevant information for RAG-based answer generation.

PostgreSQL with **pgvector** will be used to store vector embeddings along with application data and metadata. This provides a common storage layer for both conventional relational information and vector-based retrieval.

5.Database and Storage

PostgreSQL

PostgreSQL will be used as the primary relational database for storing information such as users, projects, experiments, configurations, evaluation results, and system metadata. It will provide structured and reliable data management for the different modules of AeroqAI. The database will also support efficient access to stored information and maintain consistency across the platform.

Redis

Redis will support caching and temporary data requirements, helping reduce repeated processing and improving the responsiveness of frequently accessed operations.

RabbitMQ

RabbitMQ will be used for asynchronous communication and background processing. Tasks such as evaluations, optimization experiments, and other long-running operations can be processed through queues without blocking normal application requests.

6.Containerization and Deployment

Docker

Docker will be used to package the different AeroqAI services into consistent and isolated environments. The frontend, Java backend, Python AI service, and supporting infrastructure can therefore be developed and tested in environments that are closer to the final deployment setup.

7.CI/CD

GitHub Actions

GitHub Actions will be used to automate parts of the software development and quality-assurance process. It can be used to build and test the application and trigger AeroqAI's AI quality checks as part of the development pipeline.

This is particularly relevant to the project because **quality regression detection is one of AeroqAI's core features**. The system can evaluate a new version against a baseline and use the result as a quality gate.

8.Monitoring and Observability

OpenTelemetry

OpenTelemetry will be used to collect information about application execution, including traces and performance-related telemetry. This can help developers understand how requests move through the different AeroqAI services and identify performance or integration issues.

Observability is particularly useful in AeroqAI because the platform itself is designed to analyze and debug AI application behavior.

Functional Specifications

The proposed AeroqAI platform will provide the following functional capabilities to support the evaluation, optimization, monitoring, and debugging of LLM and RAG-based applications.

1.User Authentication and Access

The system will provide secure user authentication and access control for the platform. After signing in, users will be able to access and manage their projects, datasets, experiments, configurations, and evaluation results.

2.User and Project Management

Users will be able to create and manage separate evaluation projects. Each project will maintain its own knowledge base, evaluation datasets, RAG configurations, experiments, and results, providing a structured environment for different AI applications.

3 Document and Knowledge Base Management

Users will be able to upload and manage documents that form the knowledge base for a RAG application. The system will process these documents into searchable text chunks that can be used during retrieval.

4 Evaluation Dataset Management

The platform will allow users to create and manage evaluation datasets containing questions, expected answers, and optional reference contexts. These datasets will provide a consistent basis for evaluating and comparing different RAG configurations.

5.RAG Configuration

Users will be able to configure important RAG parameters, including chunk size, chunk overlap, embedding model, Top-K retrieval, reranking, prompts, and generation settings. This allows different configurations to be tested according to the requirements of a particular application.

6.RAG Execution

The system will execute user queries through the configured RAG pipeline. Relevant information will be retrieved from the knowledge base and provided as context to the selected LLM, which will generate the final response.

7.Automated Response Evaluation

AeroqAI will automatically evaluate generated responses using configurable evaluation metrics. The system will provide individual metric scores along with an overall evaluation score, allowing users to assess response quality in a consistent manner.

8.RAG Configuration Optimization

The platform will automatically test multiple RAG configurations against the same evaluation dataset. The results will be compared to identify and recommend the configuration that provides the best overall performance.

9 Experiment Tracking and Comparison

Each experiment will be recorded along with its configuration, dataset, evaluation metrics, latency, cost, and timestamp. Users will be able to compare experiments to understand how changes in configuration affect the performance of the AI application.

10.Background Evaluation and Processing

Long-running operations such as evaluations and optimization experiments will be handled through asynchronous processing. This will allow users to initiate these tasks without having to keep the main application request active until the operation is completed. The project's architecture already specifies RabbitMQ for asynchronous experiment and evaluation workloads.

11.Quality Regression Detection

The system will compare a new evaluation run against a selected baseline to identify whether the quality of the AI application has decreased beyond a predefined threshold. This will help detect unintended quality degradation when models, prompts, or configurations are changed.

12.CI/CD Quality Gate

Based on predefined quality requirements, AeroqAI will generate a **PASS or FAIL** decision for an evaluation run. This result can be integrated into CI/CD workflows so that AI quality checks become part of the software development and deployment process.

13.Interactive RAG Debugging

The platform will provide an execution trace for individual RAG queries, including the retrieved chunks, similarity scores, reranking results, prompt, generated answer, and evaluation results. This will help developers understand where problems may have occurred during the retrieval or generation process.

14.Configuration and Policy Management

The system will maintain configurable policies and evaluation conditions that determine how experiments, quality checks, and regression decisions are performed. This will allow the evaluation process to be adapted to the requirements of different projects.

15.Dashboard and Visualization

The web dashboard will present evaluation scores, experiment comparisons, quality trends, latency, cost, regression results, and retrieval information in an accessible format. This will give users a single place to monitor and understand the performance of their AI applications.

16.API and Service Integration

The platform will expose REST APIs to connect the frontend with the application backend and AI evaluation service. This will allow the different components of AeroqAI to work together while keeping the application and AI workloads separated into their respective services. The current architecture specifies Spring Boot REST APIs and a separate Python/FastAPI AI layer.

Expected Outcome

The proposed project is expected to result in a unified and practical platform for evaluating, improving, and monitoring LLM- and RAG-based applications. AeroqAI will provide developers with a structured way to assess AI systems and make informed improvements rather than relying mainly on manual testing.

The expected outcomes of the project are:

- Systematic AI Evaluation: Enable developers to evaluate AI-generated responses using a consistent evaluation dataset and automated, semantic-based metrics.
- Improved RAG Performance: Help identify RAG configurations that provide better response quality through systematic configuration testing and comparison.
- Reliable Experiment Management: Maintain detailed records of experiments, configurations, metrics, latency, and cost so that results can be compared and reproduced.
- Early Detection of Quality Issues: Identify quality regressions by comparing new versions of an AI application with established baselines before deployment.
- Better RAG Debugging: Provide visibility into the retrieval and generation process, helping developers understand possible reasons behind incorrect or low-quality responses.
- CI/CD Integration: Enable AI quality checks to become part of the software development process through automated quality gates and PASS/FAIL decisions.
- Centralized Monitoring: Provide a single dashboard for viewing evaluation results, experiment comparisons, quality trends, retrieval information, latency, cost, and regression results.

Overall, the project will demonstrate how RAG, vector search, automated LLM evaluation, configuration optimization, experiment tracking, regression testing, debugging, and CI/CD quality checks can be brought together within a single platform. The expected result is a more structured, repeatable, and measurable approach to AI quality assurance, helping developers understand not only whether an AI application performs well, but also where improvements may be needed.

Future Scope

The proposed AeroqAI platform provides a foundation for systematic evaluation and quality improvement of LLM- and RAG-based applications. Once the core platform is established, it can be further enhanced to support more advanced AI evaluation, optimization, security, and production-monitoring requirements.

1.Multi-LLM Provider Support

The platform can be extended to support a wider range of commercial and open-source LLM providers. This would allow developers to compare models based on response quality, latency, and cost and select the most suitable model for a particular application.

2.Advanced RAG Optimization

The current configuration-based optimization approach can be expanded with techniques such as Bayesian optimization and evolutionary search. These approaches could reduce the number of experiments required while efficiently identifying high-performing RAG configurations.

3.Advanced Retrieval Techniques

Future versions can incorporate more sophisticated retrieval strategies, including hybrid search, BM25, advanced reranking, query rewriting, multi-query retrieval, parent-document retrieval, and context compression. These capabilities could provide better retrieval quality for more complex applications.

4.AI Agent Evaluation

The scope of AeroqAI can be extended beyond traditional RAG systems to evaluate AI agents and multi-step workflows. Future evaluation could consider tool selection, tool execution, multi-step task completion, and agent safety**.**

5.Security and Safety Evaluation

The platform can be enhanced with dedicated security and safety evaluations for AI applications. This may include testing for prompt injection, sensitive-information leakage, and unsafe or inappropriate model outputs**.**

6.Detailed Regression Analysis

Instead of reporting only an overall quality decrease, future versions could identify which questions, topics, datasets, or evaluation categories are responsible for a regression. This would make regression results more actionable for developers.

7.Production Monitoring

AeroqAI could eventually be extended to monitor AI applications in production. Continuous monitoring could track changes in response quality, latency, cost, and safety, helping teams identify issues after deployment.

8.Deeper CI/CD Integration

The platform can provide broader integration with development and CI/CD platforms such as GitHub and GitLab. Evaluation and quality checks could then be triggered automatically whenever there is a change to the model, prompt, RAG configuration, or application code.

9.Multi-Tenant SaaS Architecture

In the future, AeroqAI could evolve into a multi-tenant SaaS platform where multiple teams or organizations can independently manage their projects, datasets, experiments, and evaluation results while maintaining appropriate separation between users and projects.

10.Explainable Evaluation

Future releases can provide more detailed explanations behind evaluation scores instead of presenting only numerical results. The system could help identify why a response received a low score and which part of the RAG pipeline may have contributed to the failure.

Overall Future Direction

Overall, the future development of AeroqAI can move from a platform focused primarily on evaluation and optimization toward a more comprehensive AI quality engineering and LLMOps platform. This would allow teams to evaluate AI systems throughout their development lifecycle, from experimentation and debugging to continuous testing and production monitoring.

Conclusion

AeroqAI is proposed as a **unified platform for evaluating, improving, and monitoring LLM- and RAG-based applications**. It brings RAG execution, automated evaluation, configuration optimization, experiment tracking, regression detection, and interactive debugging into a single workflow.

The platform will help developers move beyond manual AI testing by providing **consistent evaluation, experiment comparison, quality monitoring, and CI/CD quality checks**. It will also provide visibility into the retrieval and generation process, helping developers identify possible causes of poor or incorrect responses.

A key focus of AeroqAI is to make AI quality **measurable and repeatable**. By evaluating different configurations using consistent datasets and maintaining experiment history, developers can make informed decisions when modifying models, prompts, retrieval settings, or other components of an AI application.

Overall, AeroqAI aims to provide a **structured, practical, and continuous approach to AI quality assurance** by bringing together RAG, vector search, LLM evaluation, optimization, regression testing, debugging, experiment tracking, and LLMOps in a single platform.

References

1. Lewis, P., et al. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. Advances in Neural Information Processing Systems (NeurIPS), 2020.
2. Gao, Y., et al. *Retrieval-Augmented Generation for Large Language Models: A Survey*. arXiv, 2023.
3. Zheng, L., et al. *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*. Advances in Neural Information Processing Systems (NeurIPS), 2023.
4. Es, S., James, J., Espinosa-Anke, L., and Schockaert, S. *RAGAS: Automated Evaluation of Retrieval Augmented Generation*. Proceedings of EACL, 2024.
5. Malkov, Y. A. and Yashunin, D. A. *Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs*. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2020.
6. PostgreSQL Global Development Group. *PostgreSQL Documentation*.
7. pgvector. *Open-source vector similarity search for PostgreSQL*.
8. Spring Boot Documentation. *Spring Boot Reference Documentation*.
9. FastAPI Documentation. *FastAPI — Modern, Fast Web Framework for Building APIs with Python*.
10. OpenTelemetry Authors. *OpenTelemetry Documentation*.

Approved by:

Guide
Prof. Monika Bagade